# Sistema de Inventario para los Almacenes UADY

# Nombre de la aplicación

Sistema de Trazabilidad e Inventarios UADY.

# Funcionalidad principal

- El Sistema 1 (sistema centralizado) se encarga de registrar y almacenar toda la información de los productos, generando un código QR para darles seguimiento.
- El Sistema 2 (sistema del proveedor) permite realizar peticiones para actualizar y registrar a qué tienda específica se le vendió un producto.

# Integración futura entre ambos sistemas

Ambos sistemas se conectarán utilizando la arquitectura distribuida del patrón Broker. La integración se implementará separando los procesos e interconectándolos a través de Sockets TCP, los mensajes de las solicitudes y respuestas serán empaquetados en formato JSON (mediante marshalling y unmarshalling).

El sistema soportará atención concurrente usando hilos, además de integrar registro dinámico, timeouts y manejo de desconexiones para asegurar la robustez de la comunicación entre el proveedor y el sistema central.

# Diseño general

- **Cliente (Sistema 2):** Construye la solicitud de actualización de inventario en JSON y la envía al Broker por socket, sin conocer ni instanciar de manera directa al sistema centralizado.
- **Broker (Intermediario):** Es un tercer sistema independiente, completamente separado del Sistema 1 y del Sistema 2, cuya única responsabilidad es registrar servicios y redirigir las solicitudes entre ellos. Recibe las solicitudes del Sistema 2, identifica la operación a realizar consultando su registro de servicios y reenvía la

petición al sistema correspondiente. Posteriormente recibe el resultado y devuelve la respuesta al cliente, gestionando errores básicos.

- **Proveedor (Sistema 1):** Expone una interfaz de servicio para ejecutar la operación solicitada. Procesa la actualización de la tienda y la vinculación con el QR, retornando una respuesta (que incluye el estado, el resultado y la fecha) al Broker.

# Requisitos Funcionales

## Sistema 1 (Sistema Centralizado / Proveedor)

- Registrar y almacenar la información detallada de los productos.
- Generar un código QR que contendrá la información del nombre del producto, ID en la base de datos y su número de serie, permitiendo dar seguimiento a la trazabilidad de cada producto.
- Procesar las peticiones para actualizar la tienda a la que se le vendió el producto y realizar la vinculación correspondiente con el código QR.
- Ejecutar la operación solicitada a través de una interfaz de servicio común y retornar una respuesta que incluya el estado, resultado y fecha.

## Sistema 2 (Cliente del Proveedor)

- Construir solicitudes estructuradas para registrar a qué tienda específica se le vendió un producto.
- Enviar la solicitud de actualización de inventario al Broker sin instanciar ni conocer la ubicación directa del Sistema 1.
- Recibir y presentar la respuesta de confirmación de la operación enviada por el Broker.

## Broker y Registro de Servicios

- Recibir las solicitudes emitidas por el Sistema 2 e identificar la operación a realizar.
- Mantener un registro que permita registrar proveedores, buscar servicios y sustituirlos dinámicamente sin modificar al cliente.
- Reenviar las peticiones al sistema correspondiente (Sistema 1) y devolver el resultado al cliente original.
- Gestionar errores básicos, rechazando servicios desconocidos o solicitudes con parámetros inválidos.

# Requisitos No Funcionales

## Arquitectura y Comunicación

- El sistema debe estar desarrollado en el lenguaje de programación Java.
- Debe implementarse bajo una arquitectura distribuida basada en el patrón Broker, separando el cliente, el intermediario y los proveedores en procesos distintos.
- La comunicación entre los componentes del sistema distribuido debe realizarse utilizando Sockets TCP.
- El intercambio de información de las solicitudes y respuestas debe estructurarse obligatoriamente en formato JSON.

## Rendimiento y Confiabilidad

- El sistema debe implementar marshalling y unmarshalling para el procesamiento de los mensajes JSON.
- El servidor debe soportar atención concurrente de múltiples peticiones mediante el uso de hilos.
- Debe incluir mecanismos de resiliencia de red, específicamente el manejo de timeouts, gestión de desconexiones y reintentos limitados.
- Las solicitudes y respuestas deben incluir metadatos obligatorios como el identificador único, nombre de la operación, estado, mensaje de error (si aplica) y la marca de tiempo (fecha y hora).

# Casos de Uso

Especificación funcional de interacción y arquitectura distribuida entre componentes centrales, intermediarios y clientes.

## Caso de Uso 1: Registrar Producto y Generar QR

- **Actor:** Administrador del Almacén (Sistema 1 - Centralizado)
- **Descripción:** Permite registrar la información detallada de los productos y generar un código QR que permita dar seguimiento a su trazabilidad.
- **Flujo principal:**
- Se ingresa la información del producto.
- El Sistema 1 registra y almacena la información detallada.
- Se genera un código QR vinculado al producto.

## Caso de Uso 2: Actualizar Tienda de Venta

- **Actor:** Proveedor (Sistema 2 - Cliente)
- **Descripción:** Permite registrar a qué tienda específica se le vendió un producto mediante el envío de una solicitud.
- **Flujo principal:**
- El Sistema 2 construye una solicitud estructurada en JSON.
- El Sistema 2 envía la solicitud al Broker a través de Sockets TCP.
- El Sistema 2 recibe la respuesta enviada por el Broker y la presenta al usuario.

## Caso de Uso 3: Procesamiento y Vinculación de Venta

- **Actor:** Sistema Centralizado (Sistema 1 - Proveedor)
- **Descripción:** El Sistema 1 procesa la actualización de la tienda y la vincula con el código QR correspondiente.
- **Flujo principal:**
- El Sistema 1 recibe la solicitud a través de su interfaz común de servicio.
- Procesa la actualización y vincula la venta al código QR.
- Genera una respuesta incluyendo estado, resultado, y fecha y hora.

- Retorna la respuesta al Broker.

## Caso de Uso 4: Intermediación de Solicitudes y Enrutamiento

- **Actor:** Broker
- **Descripción:** El Broker recibe peticiones del Sistema 2, identifica la operación y la enruta al Sistema 1 sin que los sistemas se conozcan directamente.
- **Flujo principal:**
- El Broker recibe la solicitud del Sistema 2.
- Identifica la operación y busca al proveedor adecuado en el registro de servicios.
- Reenvía la solicitud al Sistema 1.
- Recibe el resultado y se lo devuelve al Sistema 2 original.
- Gestiona posibles errores como servicios desconocidos o parámetros inválidos.

## Caso de Uso 5: Gestión de Registro de Servicios

- **Actor:** Broker y Proveedores
- **Descripción:** Mantenimiento del registro dinámico de proveedores para poder sustituirlos sin afectar al cliente.
- **Flujo principal:**
- Los proveedores (como el Sistema 1) se registran asociando un nombre de servicio con su instancia.
- El Broker mantiene esta lista actualizada y permite localizar al proveedor correcto durante una solicitud.

# Tecnologías a utilizar

- **Lenguaje de Programación:** Java (obligatorio para el desarrollo de ambos sistemas).
- **Plataforma del Sistema del Proveedor (Sistema 2):** PC (Aplicación de escritorio en Java, utilizando Java Swing estilizado con la librería FlatLaf para la interfaz de usuario).
- **Plataforma del Sistema Centralizado (Sistema 1):** Java (Backend para la lógica de almacenamiento, generación de códigos QR y vinculación de ventas, utilizando la librería ZXing (Zebra Crossing) para generar códigos QR en formato PNG), con una interfaz de escritorio para el Administrador desarrollada en Java Swing y estilizada con FlatLaf.
- **Arquitectura de Comunicación:** Sockets TCP e Hilos (Threads) para la atención concurrente en el Broker.
- **Intercambio de Datos:** Formato JSON (implementando marshalling y unmarshalling mediante la librería Gson en ambos sistemas).
- **Despliegue y Entorno:** Docker Compose (para definir una estructura y proceso de desarrollo uniforme y más sencillo al orquestar los tres sistemas).

# Arquitectura y Estructura de Carpetas del Sistema 1 (Centralizado / Proveedor)

- **Base de Datos y Persistencia:** MariaDB para el almacenamiento de la información de los productos, utilizando el conector JDBC (MariaDB Connector/J) para la integración con el backend en Java.

**Arquitectura:** Se empleará una arquitectura por capas (Modelo-Servicio-Repositorio) que separa la lógica de negocio, el acceso a datos y la comunicación de red para mantener el sistema modular. La interfaz del Administrador se organiza con el patrón MVC sobre la capa de servicio: las vistas y controladores nunca acceden directamente a los repositorios. La interfaz solo se abre cuando hay una pantalla disponible; en entornos sin pantalla (como el contenedor Docker) el sistema arranca únicamente

como servicio.
**Estructura de Carpetas:**

- **model**: Contiene las clases de dominio (entidades como Producto, Venta y representaciones del QR).
- **service**: Contiene la lógica de negocio, incluyendo la generación de códigos QR y la vinculación de ventas.
- **repository**: Encargado del almacenamiento y acceso a la información de los productos en la base de datos.
- **network**: Implementa la conexión por Sockets TCP, exponiendo la interfaz del servicio, registrándolo en el Broker y procesando las peticiones JSON entrantes.
- **util**: Clases auxiliares para el marshalling y unmarshalling de JSON y herramientas de generación de QR.
- **view**: Ventanas del Administrador (ventana principal con pestañas de Productos, Mercancía y Ventas) desarrolladas con Java Swing y estilizadas con FlatLaf.
- **controller**: Manejadores de los eventos de la interfaz que invocan a la capa de servicio en segundo plano, sin congelar la interfaz.

# Arquitectura y Estructura de Carpetas del Sistema 2 (Cliente)

**Arquitectura:** Se empleará el patrón MVC (Modelo-Vista-Controlador) para desacoplar de forma efectiva la interfaz gráfica de escritorio de la lógica de comunicación con el intermediario.
**Estructura de Carpetas:**

- **model**: Clases de datos y DTOs (Data Transfer Objects) para estructurar las solicitudes y procesar las respuestas del inventario.
- **view**: Interfaces gráficas de usuario (ventanas, formularios de actualización) desarrolladas estrictamente con Java Swing y estilizadas con la librería FlatLaf.
- **controller**: Manejadores que capturan los eventos de la interfaz (clics, ingresos de texto) y ordenan las ejecuciones de red.
- **network**: Cliente Socket TCP que envía las solicitudes estructuradas en JSON al Broker y espera la respuesta sin conocer al Sistema 1.

- **util**: Herramientas auxiliares, dedicadas principalmente a la conversión y formateo de mensajes JSON.

# Arquitectura y Estructura de Carpetas del Broker (Intermediario)

- **model:** Clases y DTOs para representar la estructura genérica de las solicitudes y respuestas en tránsito.
- **registry:** Módulo encargado de mantener en memoria el registro dinámico de proveedores y servicios disponibles, permitiendo búsquedas, registros y validaciones rápidas.
- **network:** Manejo central de las comunicaciones de red; incluye la configuración del ServerSocket, la asignación de hilos (Threads) para la atención concurrente de múltiples peticiones y el enrutamiento de los mensajes.
- **util:** Clases auxiliares para el marshalling y unmarshalling de los mensajes JSON (utilizando Gson) y la gestión centralizada de errores o timeouts.

# Modelo de Base de Datos (MariaDB)

El sistema utiliza MariaDB para la persistencia de datos. A continuación se definen las entidades principales que soportan el registro de productos, la generación de códigos QR (que incrustan el nombre del producto, su ID en la base de datos y su número de serie) y el seguimiento de las ventas en los distintos almacenes.

## Entidades y Atributos

### ROLES

Catálogo independiente para definir jerarquías operativas.

- id_rol (PK)
- nombre_rol (ej. Administrador Sistema 1, Proveedor Sistema 2)
- descripcion

### USUARIOS

Gestiona a los actores que interactúan con los sistemas.

- id_usuario (PK)
- id_rol (FK)
- nombres
- apellido_paterno
- apellido_materno

### PRODUCTOS

Almacena la información general del catálogo de inventario.

- id_producto (PK)
- nombre
- descripcion
- precio_actual

## HISTORIAL_PRECIO

Mantiene la auditoría de las fluctuaciones de costo de los productos.

- id_historial (PK)
- id_producto (FK)
- precio_historico
- fecha_cambio

## MERCANCIA

Representa cada unidad física individual para su trazabilidad exacta.

- numero_serie (PK)
- id_producto (FK)
- estado (ej. en almacén, vendido)
- ruta_qr (Ubicación del archivo PNG del código QR)

## TIENDAS_ALMACENES

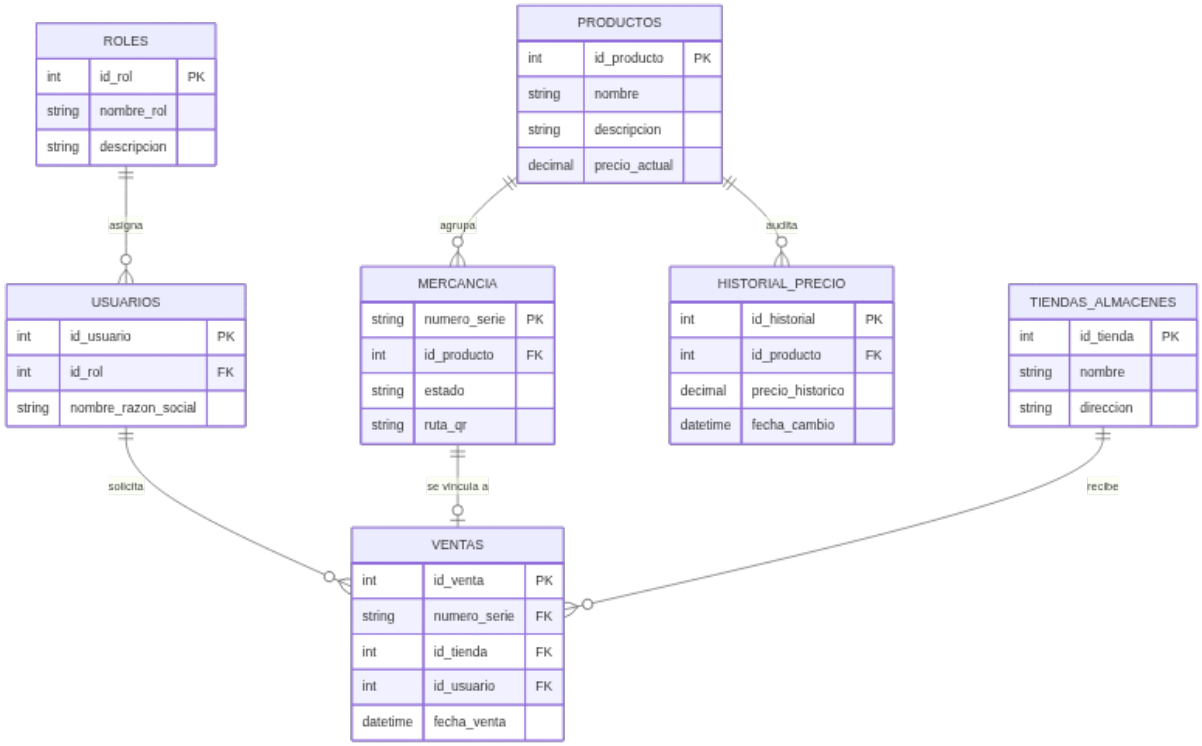
Catálogo de los destinos físicos o sucursales de la UADY.

- id_tienda (PK)
- nombre
- direccion

## VENTAS

Registra la petición de actualización y vincula la mercancía a su destino final.

- id_venta (PK)
- numero_serie (FK)
- id_tienda (FK)
- id_usuario (FK)
- fecha_venta


ROLES
int id_rol PK
string nombre_rol
string descripcion
PRODUCTOS
int id_producto PK
string nombre
string descripcion
decimal precio_actual
asigna
agrupa
audita
USUARIOS
int id_usuario PK
int id_rol FK
string nombre_razon_social
MERCANCIA
string numero_serie PK
int id_producto FK
string estado
string ruta_qr
HISTORIAL_PRECIO
int id_historial PK
int id_producto FK
decimal precio_historico
datetime fecha_cambio
TIENDAS_ALMACENES
int id_tienda PK
string nombre
string direccion
solicita
se vincula a
recibe
VENTAS
int id_venta PK
string numero_serie FK
int id_tienda FK
int id_usuario FK
datetime fecha_venta

# Contrato de interacciones

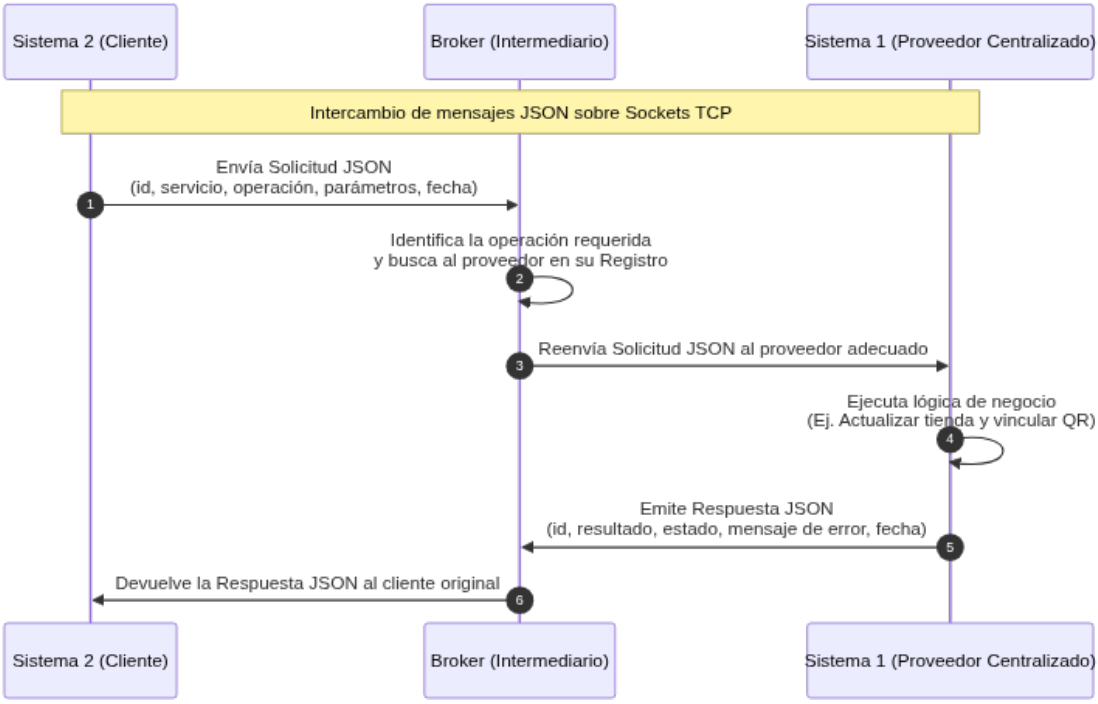
La comunicación entre los sistemas se realizará mediante mensajes en formato JSON transmitidos a través de Sockets TCP.

## Atributos de la Solicitud

- **Identificador de solicitud:** ID único para rastrear el mensaje.
- **Nombre del servicio:** Nombre del servicio solicitado al Broker.
- **Nombre de la operación:** Acción específica a ejecutar.
- **Parámetros:** Datos requeridos para la operación (ej. numero_serie, id_tienda, id_usuario).
- **Fecha y hora:** Marca temporal del envío de la solicitud.

## Atributos de la Respuesta

- **Identificador de solicitud:** ID correspondiente a la petición original.
- **Resultado:** Datos devueltos tras procesar la operación.
- **Estado:** Estado de ejecución (ej. éxito o error).
- **Mensaje de error:** Descripción del error (si corresponde).
- **Fecha y hora:** Marca temporal de la respuesta.


Sistema 2 (Cliente)
Broker (Intermediario)
Sistema 1 (Proveedor Centralizado)
Intercambio de mensajes JSON sobre Sockets TCP
Envía Solicitud JSON
(id, servicio, operación, parámetros, fecha)
1
Identifica la operación requerida
y busca al proveedor en su Registro
2
Reenvía Solicitud JSON al proveedor adecuado
3
Ejecuta lógica de negocio
(Ej. Actualizar tienda y vincular QR)
4
Emite Respuesta JSON
(id, resultado, estado, mensaje de error, fecha)
5
Devuelve la Respuesta JSON al cliente original
6
Sistema 2 (Cliente)
Broker (Intermediario)
Sistema 1 (Proveedor Centralizado)